COMP2271: Computer Graphics

Part 5: Build a World

Lecturer: Frederick Li
Department of Computer Science
Durham University

How to Learn This Lecture

This module is about structure and relationships. To master the Scene Graph, think like an architect:

- **Think in Relatives:** Stop thinking about absolute world coordinates (e.g., "The hand is at x=50"). Start thinking in local space (e.g., "The hand is at x=0 relative to the arm").
- **The Multiplication Chain:** Visualise the transformation flow. A child's final position is the result of multiplying its local matrix by its parent's world matrix. This chain reaction ($Child \cdot Parent \cdot Grandparent...$) is the core mechanic.
- **Separation of Concerns:** Understand that the complexity lives on the CPU (traversing the tree). The GPU remains "dumb"; it just receives a single, final matrix for each object, completely unaware of the hierarchy.

1 Introduction: The Challenge of Complexity

Up to this point, our journey in computer graphics has focused on the fundamental task of rendering individual objects. We have learned how to define their geometry, apply transformations to position them, and simulate light to give them a three-dimensional appearance. However, any meaningful 3D world, be it in a video game, a simulation, or an architectural visualisation, is composed of not one, but hundreds or thousands of objects.

Hierarchies are natural in our world; a person has arms, which have hands, which have fingers. A car has wheels, which have lug nuts. A solar system has planets, which have moons. A simple approach of managing each object in a flat list quickly becomes unmanageable. Consider a car model: the chassis, four wheels, doors, and steering wheel are all distinct objects. If we want to move the car forward, we would have to apply the exact same translation to every single component. If we want the wheels to spin while the car moves, the calculations become even more complex.

How do we manage these intricate relationships in a way that is both intuitive and computationally efficient? This module introduces the solution: the **Scene Graph**. It is the fundamental data structure that provides a robust and scalable framework for organising all the elements of a 3D scene, transforming a chaotic collection of objects into a structured, hierarchical world.

2 The Scene Graph: A Hierarchical Worldview

2.1 Core Definition

Scene Graph

A **Scene Graph** is a hierarchical data structure, specifically a *directed acyclic graph* (DAG), used to organise and manage the objects within a 3D scene. Every element in the scene, such as a 3D model, a light source, or a camera, is represented as a **node** in this tree-like structure.

The term *Directed Acyclic Graph* means:

- **Directed:** The relationships have a clear direction, from parent to child. You cannot go backwards.
- **Acyclic:** A node cannot be its own ancestor. This prevents infinite loops in transformations (e.g., a hand cannot be the parent of the arm it is attached to).

The power of a scene graph stems from its use of **parent-child relationships**. Each node in the graph can have one parent and multiple children. This hierarchy has a profound implication for transformations:

- Each node stores its own transformation (position, rotation, and scale) **relative to its parent node**. This is known as its **local transformation** and exists in its own **local space** (or object space).
- The final transformation that places an object in the 3D world is called its **world transformation**. This is calculated by combining the node's local transformation with the world transformation of its parent. This final state exists in **world space**.

This process of combining matrices continues up the hierarchy until we reach the **root node**, which represents the origin of the entire scene or "world space".

2.2 Concatenating Transformations: The Mathematics

The process of calculating a node's final world transformation involves traversing up the scene graph and multiplying the local transformation matrices at each step. Let M_{local} be the local transformation matrix of a node (containing its relative position, rotation, and scale). Let M_{parent_world} be the already calculated world transformation matrix of its parent. The node's own world transformation, M_{world}, is then:

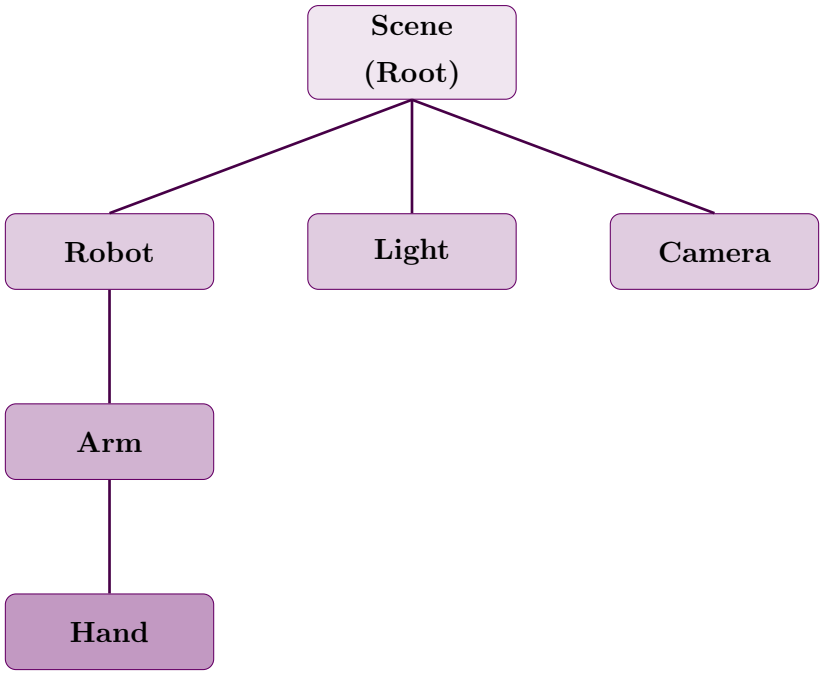


Figure 1: A diagram of a simple Scene Graph. The Robot, Light, and Camera are children of the Scene's root. The Arm is a child of the Robot, meaning any transformation applied to the Robot will also affect the Arm.

$$M_{world} = M_{parent\_world} \times M_{local}$$

This calculated M_{world} is the **Model Matrix** we send to the vertex shader. The GPU does not need to know about the scene graph's complexity; it only needs the final resulting matrix for each object it renders. This is a powerful abstraction that keeps our shader code simple while allowing for complex hierarchical arrangements on the CPU.

2.3 Connecting to Core Concepts

The scene graph is not a new, isolated concept; rather, it is a practical and essential application of the principles we learned in **Module 2: Transformations**.

- **Model Matrix:** The entire purpose of traversing the scene graph for a given object is to compute its final Model Matrix for the MVP pipeline. The hierarchy provides an intuitive way to build this matrix from a series of simpler, relative transformations.
- **Hierarchical Animation:** The structure makes complex animations, like a walking character, manageable. Animators only need to define the local rotations of the leg joints. The scene graph automatically computes the final world positions of the lower legs and feet based on the transformations of the upper legs and torso.

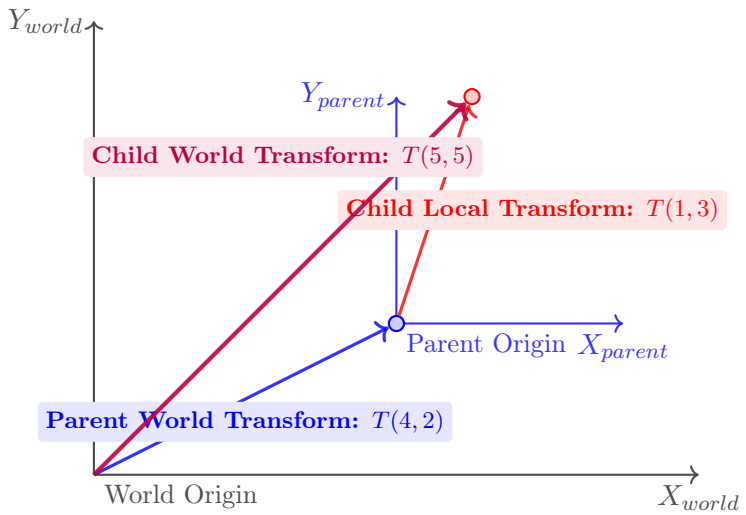


Figure 2: Visualising concatenated transformations. The Parent is translated to (4,2) in world space. The Child is translated to (1,3) in the Parent's local space. Its final world position is the combination of both transformations, resulting in (5,5).

3 Practical Scenarios and Applications

3.1 Scenarios of the Theory

To understand the power of a scene graph, consider these practical examples:

- **A Solar System:** The Sun is placed at the root of the scene. The Earth is added as a child of the Sun, with a local translation matrix that defines its orbital radius. The Moon is then added as a child of the Earth, with its own local translation for its orbit. When you apply a rotation to the Sun, nothing happens to the others. When you apply an orbital rotation to the Earth node, the Moon automatically follows, perfectly maintaining its own orbit around the moving Earth.
- **A Vehicle:** A car's chassis is a parent node. The four wheels are children of the chassis. When you apply a forward translation to the chassis, all wheels move with it. However, you can still apply an independent local rotation to each wheel node to make them spin, without affecting the chassis. This cleanly separates the vehicle's movement from the wheels' rotation.
- **Graphical User Interfaces (GUIs):** A window is a parent node. It may contain child nodes for a title bar and a content panel. The content panel can, in turn, be a parent to buttons and text boxes. If you drag the window, the entire hierarchy of child elements moves with it seamlessly, because their positions are all relative to the window's top-left corner.

3.2 Primary Applications

The scene graph is not an abstract theoretical concept; it is the backbone of virtually all modern 3D applications.

- **Game Engines:** Engines like Unity, Unreal Engine, and Godot are fundamentally built around the scene graph concept. The "hierarchy" or "outliner" panel in these editors is a direct visualisation of the scene graph.
- **3D Modelling Software:** Applications like Blender, Maya, and 3ds Max use a scene graph to manage the complex relationships between objects, modifiers, skeletons, and constraints.
- **Web Graphics Libraries:** High-level libraries such as Three.js are scene graph-based. The ‘scene.add(object)‘ and ‘parent.add(child)‘ methods are direct manipulations of the scene graph hierarchy.

4 Implementation in Practice

While the vertex shader remains simple, the complexity of the scene graph is managed on the CPU side, typically in JavaScript for WebGL applications.

4.1 Calculating a Child’s World Matrix

This conceptual code shows the core logic. To get a child’s final world matrix, you must multiply its local transformation by its parent’s world transformation.

```
// Pseudo-code using a matrix library like gl-matrix
// Assume the parent's world matrix has already been computed
const parentWorldMatrix = getParentWorldTransform();
// Create the child's local matrix (relative to the parent)
const childLocalMatrix = mat4.create();
// Start with identity
// Move the child 5 units along the parent's X-axis
mat4.translate(childLocalMatrix, childLocalMatrix, [5, 0, 0]);
// The child's world matrix is the parent's world * child's local
const childWorldMatrix = mat4.create();
mat4.multiply(childWorldMatrix, parentWorldMatrix, childLocalMatrix);
// 'childWorldMatrix' is now ready to be sent to the shader as the
// u_modelMatrix uniform for the child object.
```

Analysis: This is the fundamental calculation at the heart of a scene graph. The child's final world position ('childWorldMatrix') is not set independently. Instead, it is derived from the parent. If the parent's position or orientation changes in the next frame, 'parentWorldMatrix' will be different. When this code is re-run, 'childWorldMatrix' is automatically updated, ensuring the child correctly follows the parent.

4.2 The Render Loop

When drawing a scene, objects must be updated and rendered in a specific order. A parent's world matrix must be calculated before any of its children's matrices can be determined. This naturally leads to a recursive, depth-first traversal of the graph.

```
// A recursive function to update and draw the scene graph
function drawNode(node, parentWorldMatrix) {
    // 1. Calculate this node's world matrix by combining its local
    //    matrix with its parent's world matrix.
    const nodeWorldMatrix = mat4.create();
    mat4.multiply(nodeWorldMatrix, parentWorldMatrix, node.localMatrix);

    // 2. If this node has geometry, draw it
    if (node.geometry) {
        // Set the model matrix uniform for this specific node
        gl.uniformMatrix4fv(modelMatrixLoc, false, nodeWorldMatrix);
        // Execute the draw call for this node's geometry
        drawObject(node.geometry);
    }

    // 3. Recursively call this function for all children
    //    This is a depth-first traversal of the scene graph tree.
    for (const child of node.children) {
        drawNode(child, nodeWorldMatrix);
        // Pass our world matrix as their parent matrix
    }
}

// In the main render loop:
const rootMatrix = mat4.create();
// Start with an identity matrix for the scene root
drawNode(scene.root, rootMatrix);
```

Analysis: This recursive approach elegantly handles any level of nesting. The ‘drawNode‘ function first computes its own world matrix using the matrix passed down from its parent. It then draws itself. Finally, and crucially, it iterates through its children and calls ‘drawNode‘ on them, passing its own newly computed world matrix as the ‘parentWorldMatrix‘ for the next level of the hierarchy.

4.3 The Unchanged Vertex Shader

One of the most elegant aspects of the scene graph is that it is a CPU-side abstraction. The GPU’s task, and therefore the shader code, remains simple and unaware of the hierarchy.

```
// Vertex Shader
uniform mat4 u_projectionMatrix;
uniform mat4 u_viewMatrix;
uniform mat4 u_modelMatrix; // This is the final calculated world matrix
attribute vec4 a_position;
void main() {
  // The GPU's job is always the same: transform a vertex
  // using the final matrices it receives.
  gl_Position = u_projectionMatrix * u_viewMatrix * u_modelMatrix * a_position;
}
```

Analysis: This separation of concerns is critical for efficient graphics engine design. The CPU handles the logical scene structure and computes the final model matrix for each object by traversing the graph. The GPU then receives this final matrix and performs what it does best: applying that transformation to thousands of vertices in parallel. The shader doesn’t care if the matrix came from a simple object or a deeply nested child node.

5 Bridging Theory and Practice: Three.js

While understanding the manual matrix calculations is crucial, high-level libraries like Three.js abstract this complexity away, providing an intuitive API that directly mirrors the scene graph concept. It handles the traversal and matrix concatenation for us, letting us focus on the creative aspects of building a scene.

5.1 Fully-Explained Three.js Example: A Solar System

Here is a practical example of building a simple solar system, which perfectly demonstrates the power of the scene graph hierarchy.

```
// 1. SETUP: Scene, Camera, Renderer (Boilerplate)
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75, w / h, 0.1, 1000);
camera.position.z = 30;
const renderer = new THREE.WebGLRenderer();
// 2. CREATE NODES: Each mesh is a node in our scene graph
const sun = new THREE.Mesh(sphereGeom, sunMaterial);
const earth = new THREE.Mesh(sphereGeom, earthMaterial);
const moon = new THREE.Mesh(sphereGeom, moonMaterial);
// 3. BUILD THE HIERARCHY (The Scene Graph)
// The `scene` object is the root node. Add the sun to it.
scene.add(sun);

// Add the earth as a CHILD of the sun. Its position is now
// relative to the sun's center.
sun.add(earth);

// Add the moon as a CHILD of the earth. Its position is now
// relative to the earth's center.
earth.add(moon);

// 4. SET LOCAL TRANSFORMATIONS
// Position the earth 10 units away from the sun
earth.position.x = 10;
// Position the moon 2 units away from the earth
moon.position.x = 2;
// 5. ANIMATION LOOP (Applies transformations each frame)
function animate() {
    // Rotate the sun on its own axis.
    // Because the earth is a child, it will orbit the sun.
    sun.rotation.y += 0.005;
    // Rotate the earth on its own axis.
    // Because the moon is a child, it will orbit the earth.
    earth.rotation.y += 0.01;

    // Render the entire scene. Three.js will automatically
    // traverse the graph, calculate all world matrices, and
    // issue the draw calls.
    renderer.render(scene, camera);
```

```
    requestAnimationFrame(animate);
}
```

Analysis: This code perfectly illustrates the power of the scene graph abstraction.

- **Node Creation:** Every ‘THREE.Mesh‘ or ‘THREE.Object3D‘ is a node.
- **Hierarchy:** The ‘parent.add(child)‘ method creates the parent-child relationship, forming the edges of the graph.
- **Local Transforms:** Modifying ‘.position‘ or ‘.rotation‘ changes the node’s *local* transformation matrix.
- **Concatenation:** When ‘renderer.render()‘ is called, Three.js traverses this graph. For the moon, it calculates its world matrix by multiplying the sun’s world matrix by the earth’s local matrix, and then by the moon’s local matrix.

By parenting the Earth to the Sun using ‘sun.add(earth)‘, we get the orbital motion "for free" just by rotating the Sun. The library handles the matrix concatenations behind the scenes. The ‘.position‘, ‘.rotation‘, and ‘.scale‘ properties are user-friendly controls that the engine uses to build the ‘localMatrix‘ for each object before traversing the graph and rendering the scene.

6 Applications of the Scene Graph

The Scene Graph is not merely a storage container for objects; it is a functional structure that solves several distinct problems in computer graphics simultaneously. While its primary mechanism, propagating transformations from parent to child, is simple, it enables four critical applications: **Construction**, **Animation**, **Optimisation**, and **Management**.

6.1 Model Construction (Composition)

Complex objects are rarely modelled as a single, rigid mesh. Instead, they are composed of separate, modular parts arranged in a hierarchy.

- **The Concept:** A "Car" object is not a monolith; it is a structural group containing a chassis, four independent wheels, doors, and a steering wheel.
- **The Benefit:** If we hard-coded the global coordinates of every vertex in the car, moving the car forward would require recalculating the position of every single part manually. By making the wheels children of the chassis, they only need to store their **local offset** (e.g., "1 meter left, 0.5 meters down"). When the chassis moves (M_{parent}), the wheels (M_{child}) inherit this spatial transformation automatically.
- **Independent Motion:** Furthermore, this modularity allows individual parts to have their own local animations. A wheel can spin around its own local X-axis while simultaneously hurtling down a highway in World Space.
- **Mathematical Formulation:** The final world position of the wheel is the product of the car's movement in the world and the wheel's placement on the car.

$$M_{world\_wheel} = M_{world\_chassis} \cdot M_{local\_wheel}$$

6.2 Animation (Forward Kinematics)

Animation in 3D is rarely a matter of moving isolated points; it is about articulating connected joints. This hierarchical movement, where the motion of a parent drives the motion of its descendants, is known as **Forward Kinematics (FK)**.

- **The Concept:** Consider a robotic arm or a human skeleton. Rotating the "Shoulder" joint must inherently move the "Elbow," which in turn moves the "Hand."

- **The Mechanism:** Without a scene graph, calculating the exact trajectory of the hand as the shoulder swings would require complex trigonometry. With a scene graph, the animator simply applies a rotation to the Shoulder node. The transformation matrix naturally propagates down the tree via the **chain rule of transformations**.
- **The Chain Rule:** To find the world position of the hand, the engine multiplies the matrices from the root down to the leaf:

$$M_{world\_hand} = M_{shoulder} \cdot M_{elbow} \cdot M_{wrist} \cdot M_{local\_hand}$$

6.3 Optimisation (Frustum Culling)

Modern scenes contain millions of polygons. Sending all of them to the GPU every frame, especially those completely behind the camera, would paralyse performance. One of the most powerful uses of a scene graph is to accelerate rendering by aggressively skipping unseen objects.

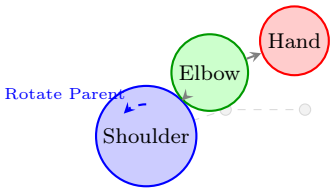
- **The Problem:** Checking if a specialised, high-poly object (like a single screw on a car engine) is visible to the camera is computationally expensive. If we check every object in a flat list individually, the time taken scales linearly ($O(N)$), which is too slow for massive scenes.
- **The Solution (BVH):** Scene graphs naturally facilitate a **Bounding Volume Hierarchy (BVH)**. We wrap parent nodes in a simple, mathematically cheap "Bounding Volume" (like a box or a sphere) that entirely encapsulates all of its children.
- **The Algorithm:**
  1. **Evaluate the Root:** Check the bounding box of the parent node (e.g., The entire Car). Is it intersecting the camera's viewing frustum?
  2. **If No (Culled):** The algorithm stops immediately for this branch. If the car cannot be seen, neither can the engine, the wheels, or the screws. We cull the entire branch in a single check.
  3. **If Yes (Recurse):** Only if the parent is visible do we traverse deeper into the tree to check the individual bounding boxes of the children. This spatial pruning reduces performance costs from $O(N)$ down to roughly $O(\log N)$.

6.4 Scene Management

Beyond physical space, the scene graph provides a logical, manageable structure for game logic and application state.

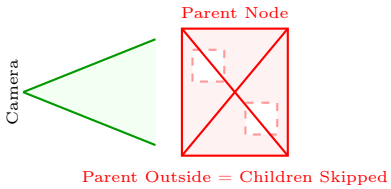
- **Grouping & Organisation:** We can group objects logically rather than physically. For example, a developer might create abstract parent nodes named "UI Elements", "Dynamic Lights", or "Enemies".
- **Batch Operations:** This structure allows for highly efficient batch processing. To pause all enemy movement, mute their sounds, or hide them entirely, a developer simply toggles a property on the "Enemies" parent node. The scene graph's traversal algorithms ensure this state change propagates instantly to hundreds of children.
- **Rendering Control:** Grouping is heavily used in the rendering pipeline. By placing all glass objects under a "Transparent" node, the engine can easily configure itself to draw opaque objects first, and then traverse the transparent branch later to ensure proper alpha blending.

A. Animation (Kinematics)



Motion propagates down the chain.

B. Optimisation (Culling)



Hierarchy enables rapid culling.

Figure 3: Visual comparison of two key scene graph applications.

Table 1: Comparison of Scene Graph Applications

Application	Concept & Role	Key Benefit
Construction	Building complex objects from simpler parts (e.g., Car Chassis + Wheels).	Relative positioning simplifies modelling.
Animation	**Forward Kinematics:** Moving a parent node automatically moves all children.	Complex motion (like a walking character) becomes manageable.
Optimisation	**Frustum Culling:** Using Bounding Volume Hierarchies (BVH) to skip invisible branches.	Reduces checks from linear $O(N)$ to logarithmic $O(\log N)$.
Management	Organizing entities into logical groups (e.g., "Lights", "NPCs").	Enables batch operations (toggling visibility, filtering).

7 Reflective Questions: Managing the Multiverse

As we conclude our exploration of Scene Graphs and hierarchies, consider the following questions. They are designed to push your understanding beyond simple parent-child relationships and into the realm of complex engine architecture.

- **The Inverse Problem (Forward vs. Inverse Kinematics):** We learned how to rotate a shoulder, then an elbow, then a wrist to place a hand (Forward Kinematics). *But what if a character needs to grab a door handle? You know where the hand needs to end up (the handle's position), but you don't know the angles for the shoulder and elbow. How would you mathematically solve the hierarchy "backwards" to find the correct joint rotations? (Search term: Inverse Kinematics / IK).*

- **The Hidden Cost (Performance):** Traversing a tree and multiplying matrices for every object, every frame, is expensive. *If you have a forest with 10,000 static trees, does it make sense to calculate 10,000 matrix multiplications every 16 milliseconds, even though the trees never move? How could we "bake" these world transformations once to save CPU cycles? (Search term: Static Batching).*

- **The Spatial Query (Collision Detection):** A Scene Graph organises objects logically (e.g., arm attached to body). *However, if a bullet is fired across the map, it doesn't care about logical parents; it cares about physical proximity. Checking the bullet against every object in the scene graph is too slow ($O(N)$). What kind of alternative data structure organises objects based on their location in space rather than their logical relationship? (Search term: Octrees / BVH).*

- **The Deep Nesting Trap (Floating Point Errors):** We calculate world positions by multiplying matrices down the chain. *If you have a solar system simulation where a moon is a child of a planet, which is a child of a star, which is a child of a galaxy... and the galaxy is at coordinate 10^{20}, what happens to the precision of the moon's movement? Computers have limited precision for floating-point numbers. How do open-world games prevent "jittering" when the player is far from the origin? (Search term: Floating Origin / Double Precision).*

"The art of programming is the art of organising complexity, of mastering multitude and avoiding its bastard chaos as effectively as possible."

— Edsger W. Dijkstra